Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра системного программирования ИСП РАН
Лаборатория (laboratory name)

Выпускная квалификационная работа бакалавра

Применение многопоточного кольцевого буфера для организации высокоскоростного обмена диагностической информацией в среде виртуализации

Автор:

Студент Б05-232 группы

Абубакиров Владимир Андреевич

Научный руководитель:

Кандидат физико-математических наук

Камкин Александр Сергеевич

Научный консультант:

научная степень

Скородумов Дмитрий Павлович

Москва 2026

Аннотация

Применение многопоточного кольцевого буфера для организации высокоскоростного обмена диагностической информацией в среде виртуализации

Абубакиров Владимир Андреевич

Современные платформы виртуализации реализуют сложный набор подсистем, обеспечивающих взаимодействие между гостевыми и хостовыми операционными системами. Однако для задач низкоуровневой отладки, профилирования и формирования корректных отчётов об ошибках критически важным является наличие высокоэффективного канала передачи диагностических данных из гостевой операционной системы на хост. В рамках работы был разработан подобный механизм, основанный на lock-free кольцевом буфере в общей памяти.

Содержание

1 Введение **4**

2 Архитектура виртуальной машины stratovirt **10**

3 Драйверы в среде UEFI **11**

4 Драйверы в операционной системе Windows **12**

5 Реализация механизма сбора диагностической информации **13**

6 Заключение **22**

1 Введение

Развитие информационной инфраструктуры, начиная от облачных платформ и центров обработки данных, заканчивая персональными компьютерами, всё чаще полагается на технологии виртуализации.

Виртуализация — группа технологий, основанных на преобразовании формата или параметров программных или сетевых запросов к компьютерным ресурсам с целью обеспечения независимости процессов обработки информации от программной или аппаратной платформы информационной системы. [1]

Простые пользователи используют т.н. виртуальные машины для запуска альтернативных операционных систем на своих рабочих станциях для тестирования подозрительных программ, для разработки и обучения. Облачные провайдеры используют виртуальные машины для обеспечения работы серверных платформ: в частности, для предоставления пользователям возможности аренды вычислительных ресурсов. Последним это позволяет не покупать собственное оборудование для разворачивания инфраструктуры, масштабировать мощности под нагрузку и платить только за фактическое использование ресурсов.

Виртуальная машина — виртуальная вычислительная система, которая состоит из виртуальных устройств обработки, хранения и передачи данных и которая дополнительно может содержать программное обеспечение и пользовательские данные. [2]

Отдельным фактором, влияющим на рост спроса на технологии виртуализации, является развитие машинного обучения и искусственного интеллекта. Развертывание и обучение моделей требует больших вычислительных ресурсов (мощных GPU и NPU). Облачные провайдеры предоставляют виртуальные машины с таким оборудованием, позволяя бизнесу и частным поль-

[1] ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

[2] ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

зователям проводить исследования, не приобретая дорогостоящее оборудование. Это дополнительно увеличивает потребность эффективного использования аппаратного обеспечения, что также достигается при помощи технологий виртуализации.

Развитие технологий виртуализации предъявляет повышенные требования к инструментам диагностики и отладки, которые должны обеспечивать сбор данных из гостевых операционных систем с минимальным влиянием на их временные характеристики. В задачах логгирования и профилирования ключевыми являются низкая задержка и высокая пропускная способность передачи диагностической информации. В связи с этим возникает необходимость создания специализированного канала связи «гость – хост», оптимизированного для однонаправленной потоковой передачи данных.

Ситуация усугубляется тем, что на одной физической рабочей станции может одновременно функционировать несколько виртуальных машин. Каждая из них генерирует свои собственные логи, операции записи которых конкурируют за общие аппаратные ресурсы. Таким образом, механизм сбора диагностических данных должен работать с минимальной задержкой не только для отдельно взятой виртуальной машины, но и масштабироваться при одновременной работе множества экземпляров.

Ещё одной особенностью современных виртуальных машин является наличие сопутствующего программного обеспечения, позволяющего пользователям бесшовно с ними взаимодействовать. Например, инструменты, обеспечивающие общий буфер обмена, перенаправление USB-устройств, синхронизацию времени и разрешения экрана. Эти системы в процессе своей работы также генерируют диагностические события, которые могут быть полезны для выявления ошибок у пользователей. Таким образом, система сбора диагностической информации должна эффективно работать с учётом функционирования нескольких самостоятельных компонентов внутри виртуальной машины.

Разработка подобной системы передачи диагностических данных под-

разумевает создание драйвера в гостевой операционной системе, который взаимодействует со специальным виртуальным устройством на стороне эмулятора. Данное устройство выступает в роле конечного приёмника сообщений, обеспечивая их сбор и последующую печать в файл. Такой подход позволяет сохранить прозрачность для пользователя, так как гостевые операционные системы воспринимают устройство как стандартное аппаратное обеспечение.

Основой данного механизма является общая память, разделяемая между гостевой операционной системой и гипервизором. Такой подход позволяет организовать канал передачи данных без применения медленных операций ввода-вывода и без дополнительных копирований между буферами.

Сложность разработки заключается в том, что в угоду минимальных задержек, синхронизация доступа к буферу осуществляется без блокировок, благодаря использованию атомарных операций, что исключает риски взаимных блокировок и обеспечивает более детерминированное время выполнения операций. Такой подход позволяет достичь высокой пропускной способности и минимальной задержки при операциях записи, что особенно важно для работы с несколькими виртуальными машинами.

Немаловажной задачей также является реализовать механизм передачи сообщений из среды BIOS (UEFI). На этом этапе происходит первичная инициализация аппаратный компонентов и сбои приводят невозможности дальнейшей загрузки системы. Отсутствие средств сбора отладочной информации существенно затрудняет выявление причин отказов.

BIOS представляет собой интерфейс между программным обеспечением и аппаратными средствами, который позволяет им общаться и взаимодействовать друг с другом. [3]

UEFI (от англ. Unified Extensible Firmware Interface, единый расширяемый микропрограммный интерфейс) был разработан консорциумом UEFI Forum на основе спецификаций, изначально предоставленных корпорацией Intel как преемника BIOS (хотя большинство образов микропрограммного обеспечения UEFI предоставляют поддержку для устаревших сервисов BIOS).

[3] Вонг А. Справочник по параметрам BIOS. — 2005

UEFI обеспечивает поддержку современного оборудования, модульной архитектуры, не зависящей от архитектуры ЦПУ и драйверов, загрузочной сети и безопасной загрузки. [4]

Важным преимуществом предлагаемого решения является возможность вывода логов от гостевой операционной системы вместе с диагностическими данными от самого эмулятора. В результате логи драйверов и сервисов, работающих в гостевых операционной системах, и сообщения эмулятора печатаются в одном месте в хронологическом порядке. Это позволяет видеть полную картину и более эффективно выявлять и исправлять ошибки. Техподдержке и разработчикам не нужно собирать диагностическую информацию из разных источников и пытаться сопоставить события по времени, ведь в одном файле доступны все логи, упорядоченные по времени.

Для решения проблем, описанных выше, предлагается механизм, основанный на использовании общей памяти. На стороне эмулятора реализуется виртуальное устройство, служащее интерфейсом для хранения метаинформации о буфере. В операционной системе разработан драйвер, обеспечивающей взаимодействие с данным устройством и передачу диагностических данных.

Объект исследования в данной работе — высокопроизводительный механизм передачи информации между гостевой операционной системой и эмулятором с минимальными временными задержками.

Предмет исследования — многопоточный кольцевой буфер без блокировок как средство передачи диагностической информации в среде виртуализации.

Целью дипломной работы является разработка комплекса программных компонентов для организации высокоскоростной передачи диагностических информации в среде виртуализации. В состав разрабатываемого решения входят: виртуальное устройства для виртуальной машины Stratovirt [5]; драйвер для среды UEFI, позволяющий передавать диагностические сообщения на самых ранних этапах работы эмулятора; а также драйвер для

[4] Энциклопедия «Касперского» — UEFI

[5] https://www.openeuler.org/en/other/projects/stratovirt/

операционной системы Windows, обеспечивающий возможность передачи диагностической информации от приложений, сервисов и других драйверов, работающих внутри гостевой системы.

Таким образом, постановка задачи заключается в разработке данных трёх компонентов. Для достижения поставленной цели требуется выполнить следующие этапы:

1. Изучить устройство и архитектуру эмулятора stratovirt.
2. Разработать виртуальное устройство на стороне эмулятора.
3. Изучение устройства и архитектуру UEFI.
4. Разработать драйвер для UEFI.
5. Разобраться в коде некоторых драйверов для ОС Windows и изучить API, предоставляемые операционной системой для разработки драйверов.
6. Реализовать драйвер для ОС Windows.
7. Сравнить реализованный канал для передачи данных с другими существующими.

Методологическую основу работы составляют: анализ технической документации и литературы о разработке драйверов для UEFI и Windows, материалы по lock-free алгоритмам и методы отладки, применяемые в среде виртуализации.

Постановка задачи определяет структуру дипломной работы, которая включает введение, четыре главы и заключение.

Введение задаёт цели и задачи работы: в этой части определяются объект и предмет исследования и раскрывается значимость.

В первой главе рассматриваются устройство и архитектура виртуальной машины stratovirt.

Вторая глава посвящена архитектуре UEFI и принципам разработки драйверов В среде UEFI.

В третьей главе описываются подходы к разработке драйверов в операционной системе Windows.

Четвёртая глава посвящена описанию реализации механизма передачи диагностических данных.

Заключение содержит обобщение результатов проведённого исследования, сравнение с другими механизмами, а также описание возможных сценариев использования разработанного решения.

2 Архитектура виртуальной машины stratovirt

3 Драйверы в среде UEFI

4 Драйверы в операционной системе Windows

5 Реализация механизма сбора диагностической информации

Разработанный механизм сбора диагностической информации включает три основных компонента:

1. Виртуальное устройства на стороне гипервизора Stratovirt.
2. Драйвер для среды UEFI.
3. Драйвер для операционной системы Windows.

Каналом взаимодействия между данными компонентами служит кольцевой буфер, расположенный в общей памяти, разделяемой между гостевой ОС и гипервизором.

Виртуальное устройство эмулируется в гипервизоре Stratovirt и выступает в роли единой точки сбора диагностической информации. Таким образом, основными функциями устройства являются:

1. Приём диагностических сообщений от драйверов гостевых систем.
2. Сериализация сообщений и запись их в файл.
3. Хранение метаинформации о буфере: его размере и типе поведения при переполнении.
4. Предоставление доступа к метаинформации посредством регистров виртуального устройства.

Драйвер для среды UEFI выполняет следующие задачи:

1. Выделение буфера в гостевой памяти типа runtime: доступной после окончания работы UEFI (после вызова ExitBootServices) и во время работы операционной системы.
2. Предоставление интерфейса для записи диагностических сообщения в буфер.
3. Регистрация обработчиков событий ExitBootServices и SetVirtualAddressMap для обеспечения работы драйвера с EfiRuntimeService.

Для упрощения интеграции с существующими UEFI модулями была разработана собственная версия DebugLib, переопределяющая стандартные функции логирования. Эти функции обращаются к вышеупомянутому драйверу для передачи сообщений.

Для обеспечения взаимодействия компонентов гостевой системы с буфером реализован драйвер, позволяющий:

1. User-space приложениям и сервисам взаимодействовать с буфером.
2. Kernel-space драйверам взаимодействовать с буфером.

Общая архитектура механизма представлена на рисунке 1.

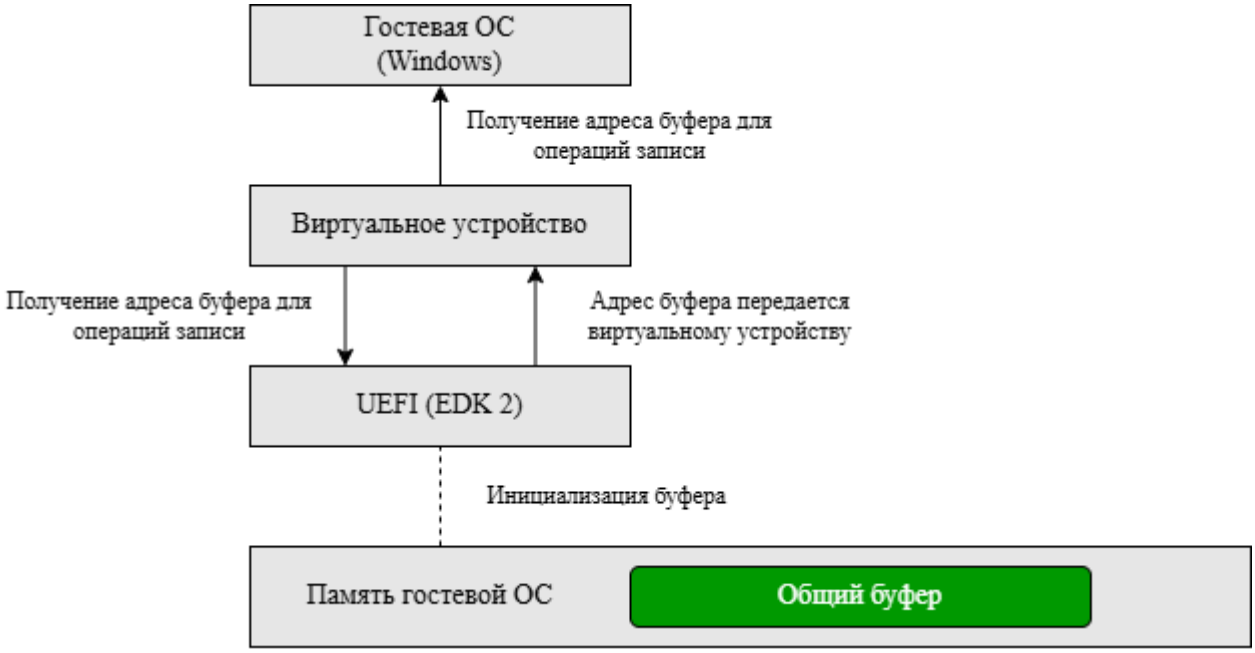

Рис. 1: Архитектура механизма обмена диагностической информацией

Виртуальное устройство может быть добавлено в устройство при помощи флага -logdevice, который принимает следующие параметры: id, flush_logs, iothread:

1. id - идентификатор виртуального устройства, позволяющий однозначно определить его в системе.
2. flush_logs - параметр логического типа, который регламентирует поведение при переполнении буфера. Если параметр установлен в true, запись сообщения будет ожидать освобождение места в буфере. Если false - сообщение, не поместившееся в буфер будет пропущено.
3. iothread - идентификатор контекста, в котором работает поток-читатель. Параметр может быть пропущен — в таком случае поток-читатель будет работать в главном контексте (main loop).

Виртуальное устройство взаимодействует с гостевыми операционными

системами через регистров, доступ к которым осуществляется посредством MMIO (memory-mapped I/O). В частности, среди прочих существует регистр, позволяющий получить информацию о поведении при переполнении буфера.

Как уже отмечалось, выделение памяти буфера выполняется драйвером UEFI. Однако на ранних стадиях работы прошивки (SEC и PEI) эта операция невозможна, т.к. память ещё не инициализирована. По этой причине, на этих этапах требуется альтернативный способ передачи диагностических данных в виртуальное устройство, поэтому в него был добавлен специальный регистр, позволяющий записываются данные последовательно — по 8, 4, 2 байта или по одному в зависимости от размера сообщения. Виртуальное устройство накапливает поступающие байты до теъ пор, пока не встретит символ окончания строки '\0', после чего сформированное сообщение записывается в выходной файл.

После инициализации памяти (на этапе PEIM) появляется возможность выделить буфер. Дальнейшая передача сообщений происходит уже через этот буфер.

Общая память организована в виде неблокирующего (lock-free) кольцевого буфера с несколькими писателями и одним читателем (MPSC — Multiple Producer Single Consumer). В начале буфера располагается заголовок, содержащий указатели текущих позиций читателя и писателя, флаг переполнения, а также вспомогательную структуру SHADOW_POS ("мнимая позиция"). Данная структура включает два поля типа UINT32: счётчик активных писателей, выполняющих копирование сообщений, и позицию резервирование места для новых сообщений.

Структура буфера представлена ниже:

```
1    typedef union {
2        struct {
3            uint32_t ReservedPos;
4            uint32_t WriterCnt;
5        } S;
6        uint64_t U;
7    } SHADOW_W_POS;
```

```

typedef struct {
    uint32_t ReadPos;
    uint32_t WritePos;
    uint64_t IsOverflow;
    SHADOW_W_POS ShWPos;
    char Buffer[0];
} LOCKLESS_RING_BUFFER;
```

Процедура записи в буфер выполняется в три этапа: резервирование места для сообщения (reserve), копирование в забронированную область (copy) и оповещение об окончании копирования (commit).

Сначала каждый поток пытается зарезервировать место для своего сообщения. Для этого каждый поток загружает текущее значение SHADOW_POS и с помощью атомарной операции compare_exchange пытается обновить его в заголовке буфера, увеличивая счётчик писателей и смещая позицию записи на размер сообщения. В случае конфликта (если compare_exchange вернула значение false) операция повторяется.

Таким образом, гарантируется, что никакой другой поток не сможет зарезервировать себе то же самое место: каждый поток копирует своё сообщение в уникальное область буфера.

Псевдокод данного этапа представлен ниже:

```
uint32_t LrbReserve (
    LOCKLESS_RING_BUFFER *Rb,
    const uint32_t MsgSize,
    SHADOW_W_POS *Curr
) {
    for (;;) {
        Curr->U = Rb->ShWPos.U;
        SHADOW_W_POS NextShadowPos = {
            .S.ReservedPos = Curr->S.ReservedPos + MsgSize,
            .S.WritePos = Curr->S.WriterCnt + 1
        };

        if (atomic_compare_exchange(&Rb->ShWPos, &Curr->U, Next.U)) {
            // SHADOW_W_POS была обновлена в буфере
```

```
15                  // Таким образом, гарантируется, что ни один другой поток
16                  // не будет копировать своё сообщение в занятую память.
17                  return Curr->S.ReservedPos;
18              }
19          }
20      }
```

После того, как поток скопировал сообщение, он сообщает об окончании операции. Поток атомарно уменьшает счётчик активных писателей в SHADOW_POS, используя аналогичный резервированию механизм синхронизации. Как только количество писателей в SHADOW_POS становится равным нулю, обновляется глобальный указатель писателя в заголовке буфера. Таким образом, указатель писателя обновляется после завершения всех операций записи, что гарантирует целостность данных в буфере и исключает чтение неполных записей на читающей стороне.

Псевдокод алгоритма представлен ниже:

```
1       uint32_t LrbCommit (
2           LOCKLESS_RING_BUFFER *Rb,
3           SHADOW_W_POS *Curr
4       ) {
5           for (;;) {
6               SHADOW_W_POS NextShadowPos = {
7                   .S.ReservedPos = Curr->S.ReservedPos,
8                   .S.WritePos = Curr->S.WriterCnt - 1
9               };
10
11              if (NextShadowPos.S.WriterCnt == 0) {
12                  // все сообщения до текущего закончили копирование
13                  Rb->WritePos = Curr->S.ReservedPos;
14              }
15              if (atomic_compare_exchange(&Rb->ShWPos, &Curr->U, Next.U)) {
16                  return;
17              }
18          }
19      }
```

После завершения записи драйвер уведомляет виртуальное устройство через отдельный регистр о том, что глобальная позиция писателя изменилась

и сообщения готовы к чтению.

Функции записи также должны проводить некоторые проверки буфера, чтобы предотвратить переполнение и запись в повреждённый буфер. Наивный алгоритм проверки мог бы выглядеть следующим образом:

```
uint32_t LrbReserve (
    LOCKLESS_RING_BUFFER *Rb,
    const uint32_t MsgSize,
    SHADOW_W_POS *Curr
) {
    for (;;) {
        Curr->U = Rb->ShWPos.U;
        uint32_t ReadPos = Rb->ReadPos;
        uint32_t BusySize = Curr->S.ReservedPos - ReadPos;
        if (BusySize > BUFFER_SIZE) {
            // Буфер сломан:
            // размер занятой области не может быть больше всего буфера.
            return -1;
        }
        if (BusySize + Size > BUFFER_SIZE) {
            // Сообщение не поместилось.
            // В зависимости от выбранного поведения, сообщение либо
            // пропускается, либо ожидает завершения записи других.
        }

        ...
    }
}
```

Однако при таком подходе возникает риск ложно принять буфер за повреждённый, если между операциями чтения структуры SHADOW_POS и проверкой целостности буфера другой поток успеет зарезервировать место в буфере, скопировать сообщение и выполнить операцию commit. Рассмотрим это на примере:

Пусть изначально:

1. Позиция читателя ReadPos = 100.
2. Аисателя WritePos = 200.
3. Количество активных писателей ShWPos.WriterCnt = 1.

При описанном ниже планировании потоков возникает указанная проблема.

Поток-писатель №1	Поток-писатель №2	Поток-читатель

(1) Операция резервирования:
Curr->U = Rb->ShWPos.U
(=> Curr->S.ReservedPos = 200)
// Поток не увеличил ShWPos.WriterCnt
// Другие потоки могут обновить WritePos		

	(2) Операция резервирования: ShWPos.U = Cur->U (=> ShWPos.S.ReservedPos = 100)	

	(3) Операция commit:
	WritePos = ShWPos.S.ReservedPos = 300
	// WritePos смог изменится, т.к.
	// поток №1 не увеличил WriterCnt	

		Читает: ReadPos = WritePos = 300
Операция резервирования: uint32_t ReadPos = Rb->ReadPos = 300 uint32_t BusySize = Curr->S.ReservedPos - ReadPos = = 200 - 300		

Таблица 1: Пример планирования потоков

Таким образом, переменная BusySize переполняется, и буфер ложно считается повреждённым.

Данную проблема решается повторной загрузкой значений Curr->U и ReadPos. Если Curr->U не изменилось, то буфер действительно повреждён. В противном случае проверку необходимо повторить.

Псевдокод проверки представлен ниже:

```
uint32_t LrbReserve (
    LOCKLESS_RING_BUFFER *Rb,
    const uint32_t MsgSize,
    SHADOW_W_POS *Curr
) {
    for (;;) {
        Curr->U = Rb->ShWPos.U;
        uint32_t ReadPos = Rb->ReadPos;
        uint32_t BusySize = Curr->S.ReservedPos - ReadPos;

        while (BusySize > BUFFER_SIZE) {
```

```
12                  uint64_t CurrCpy = Curr->U;
13
14                  // Принудительно перечитать значения ReadPos и ShWPos
15                  MemoryFence();
16                  Curr->U = Rb->ShWPos.U;
17                  ReadPos = Rb->ReadPos;
18
19                  if (Curr->U == CurrCpy) {
20                      return -1;
21                  }
22                  BusySize = Curr->S.ReservedPos - ReadPos;
23              }
24              if (BusySize + Size > BUFFER_SIZE) {
25                  // Сообщение не поместилось.
26                  // В зависимости от выбранного поведения, сообщение либо
27                  // пропускается, либо ожидает завершения записи других.
28              }
29
30              ...
31          }
32      }
```

Алгоритм потока-читателя ничем в сущности не отличается от чтения из обычного кольцевого буфера. При получении уведомления об изменении позиции писателя через регистр виртуального устройства, поток-читатель последовательно извлекает сообщения, начиная с позиции ReadPos и продвигаясь к WritePos, пока не будут прочитаны все доступные данные.

Вышеописанный алгоритм читателя реализован как в драйвере UEFI, так и в драйвере для Windows.

В драйвере для Windows были реализованы ioctl - команды для взаимодействия с буфером:

1. Для пользовательского программ (user-space): она принимает сообщение и выполняет проверки над ним и записывает его в буфер.

2. Для программ режима ядра (kernel-space) — возвращает указатель на функцию записи в буфер.

Упомянуть про особенности драйвера Windows: получение адреса из

ACPI

6 Заключение

Существуют и другие схожие решения, в частности кольцевой буфер, используемый в eBPF [2], в котором этап резервирования реализован с применением спинлоков. Такой подход обеспечивает корректность конкурентной записи, однако в условиях виртуализации и одновременной работы нескольких виртуальных машин использование блокирующих примитивов синхронизации может приводить в существенным задержкам. В случае переключения контекста внутри гостевой операционной системы удерживаемая блокировка препятствует прогрессу остальных потоков-писателей. Своё влияние на непредсказуемость задержек также оказывает планирование виртуальных процессоров на стороне гипервизора, что может существенно увеличить время удержания блокировки.

Преимуществом предложенного решения перед механизмом, используемым в Linux eBPF, является возможность отобразить кольцевой буфер непосредственно в виртуальное адресное пространство пользовательского процесса. Благодаря этому запись диагностических сообщений может выполняться без обращения к драйверу, что снижает накладные расходы на системные вызовы и переключение контекста.

Кроме того, в условиях одновременной работы нескольких виртуальных машин предложенное решение демонстрирует более предсказуемые задержки. В случае с eBPF блокирующие примитивы синхронизации (спинлоки) в сочетании с переключением контекста внутри гостевой ОС и планированием виртуальных процессоров могут приводить к более непредсказуемым задержкам.

В рамках данной работы был разработан альтернативный механизм передачи диагностических данных, основанный на неблокирующем MPSC-кольцевом буфере, размещённом в памяти гостевой операционной системы. Для синхронизации конкурентных записей используется вышеописанный lock-free алгоритм с разделением операций резервирования, копирования и фиксации записи (commit), что позволяет полностью отказаться от блокирую-

щих примитивов синхронизации. Предложенное решение обеспечивает независимый прогресс потоков и корректную работу даже при условии приостановки выполнения отдельных потоков-писателей. Реализация интегрирована в виртуализированную среду, представленную эмулятором Stratovirt, UEFI EDK2 и гостевой операционной системой Windows. Данный механизм позволяет обеспечить более предсказуемое поведение при высокой интенсивности диагностических событий.

Список литературы

[1] *Mott-Smith, H.* The theory of collectors in gaseous discharges / H. Mott-Smith, I. Langmuir // *Phys. Rev.* — 1926. — Vol. 28.

[2] *Морз, Р.* Бесстолкновительный PIC-метод / Р. Морз // Вычислительные методы в физике плазмы / Ed. by Б. Олдера, С. Фернбаха, М. Ротенберга. — М.: Мир, 1974.

[3] *Киселёв, А. А.* Численное моделирование захвата ионов бесстолкновительной плазмы электрическим полем поглощающей сферы / А. А. Киселёв, Долгоносов М. С., Красовский В. Л. // Девятая ежегодная конференция «Физика плазмы в Солнечной системе». — 2014.